# A3-PS004

Authenticated Gateway-Assisted DNSSEC for IoT Clients

**1. Prethiksha V**
**prethikshav@karunya.edu.in**

**2. Niya Cibi**
**niyacibi@karunya.edu.in**

**3. Mentor: Dr. R. Venkatesan**
**rlvenkei_2000@karunya.edu**

AIORI Hackathon 3

October 13, 2026

# Problem Description

## Authenticated DNS for constrained IoT clients, without each client running full DNSSEC validation.

- **Spoofing / cache poisoning:** a device that accepts unauthenticated DNS answers can be redirected to an attacker's endpoint.
- **Validation cost:** full DNSSEC needs extra records, crypto checks, memory, CPU, energy and extra network exchanges.
- **New trust boundary:** a gateway centralises validation and caching, so it must itself be authenticated by clients.
- **Freshness:** cached data must not outlive its TTL or validation state; bogus and indeterminate results need defined handling.
- **Evidence, not assumptions:** security and resource costs must be measured, not assumed.

**Research question: Can a trusted gateway give IoT clients authenticated DNSSEC resolution with less client-side computation and traffic, without weakening DNSSEC validation semantics?**

# Background

**DNS integrity matters for IoT: devices resolve APIs, update servers, MQTT brokers and cloud platforms by name.**

## What DNSSEC contributes [1–3]

- **DNSKEY / DS** build a chain of trust from a configured trust anchor through signed zones.
- **RRSIG** signs record sets; validators check signatures and time bounds.
- **NSEC / NSEC3** authenticate denial of existence.
- Outcomes: *secure*, *bogus*, *insecure*, *indeterminate*.

## Why gateway assistance?

- A gateway has more CPU, RAM, stable power and a maintainable software stack than a sensor node.
- It can centralise validation, reuse valid cache entries and apply per-device rules.
- But clients must authenticate the gateway, or a compromised gateway could forge results.

TLS protects the application connection, but DNS manipulation can still cause redirection attempts or expose weak endpoint validation.

# Background: Scope & Assumptions

**The prototype is scoped to authenticity, freshness, policy handling and measurable overhead.**

- **Platform:** Linux host or Raspberry Pi gateway, with simulated clients and/or ESP32-class devices.
- **No home-grown crypto:** use a standards-compliant validating resolver or library (e.g. Unbound / ldns); DNSSEC cryptography is not reimplemented.
- **Provisioning:** each client is provisioned with the gateway public key or a securely shared per-client secret.
- **Defence in depth:** application-layer TLS remains recommended alongside this design.
- **Out of scope:** a fully compromised gateway is beyond the protection of this first prototype.

Expected outcome: a working prototype and a reproducible evaluation. We do not claim that DNSSEC or gateway-based validation is new.

# Literature & RFC Review

## The design builds on existing DNSSEC standards; our contribution is the IoT-facing architecture and evaluation.

| Standard | Relevance | Design implication |
|---|---|---|
| RFC 4033: DNSSEC Intro & Requirements [1] | Goals, terminology, limitations | Use DNSSEC for origin authentication and integrity; no confidentiality claim |
| RFC 4034: DNSSEC Resource Records [2] | DNSKEY, RRSIG, DS, NSEC formats | Process records with maintained software |
| RFC 4035: DNSSEC Protocol Modifications [3] | Validation requirements | Preserve semantics; keep *bogus* distinct |
| RFC 8499: DNS Terminology [4] | Consistent vocabulary | Precise resolver, cache, TTL terms |
| RFC 7858: DNS over TLS [5] | Encrypted resolver transport | Optional upstream encryption; not data authentication |
| RFC 8767: Serving Stale Data [6] | Resiliency under defined conditions | Stale data is never treated as fresh or valid |
| RFC 8032: EdDSA [7] | Compact signatures | Candidate gateway-to-client authenticator |

# Literature & RFC Review: Gap Addressed

**The standards define DNSSEC behaviour, but not a lightweight client-facing envelope for constrained IoT devices.**

- No standard specifies a compact, authenticated *validation-result* message from a gateway to an IoT client.
- No defined IoT policy model for mapping *insecure* and *indeterminate* outcomes per device class.
- Little reproducible data comparing client cost of direct validation vs. gateway-assisted resolution on ESP32-class hardware.

**Implementation caution: a gateway-signed response authenticates what the gateway says; it does not independently prove DNSSEC validity to the client. Trust depends on correct gateway validation, secure key provisioning, freshness checks and the threat model.**

# Proposed Solution

## LADG (Lightweight Authenticated DNSSEC Gateway): validate upstream, apply policy, return a compact authenticated answer.

1. **Query:** the IoT client sends a hostname + record type and a request ID to the local gateway.
2. **Identify:** the gateway maps the request to a registered client or device class.
3. **Cache:** reuse an entry only while its TTL and validation expiry are still valid.
4. **Resolve & validate:** on a miss, query a DNSSEC-capable resolver and validate with maintained software.
5. **Apply policy:** accept *secure*; reject *bogus*; handle *insecure / indeterminate* by explicit rule.
6. **Authenticate:** build a canonical envelope (qname, qtype, answer, status, expiry, request ID) and sign or MAC it.
7. **Verify:** the client checks the authenticator, query binding and freshness before using the answer.

# Proposed Solution: Authentication & Failure Handling

**Two separate security layers: DNSSEC validation at the gateway, and gateway-to-client authentication.**

## Authenticator options

- **Ed25519 signature [7]:** clients verify with a provisioned gateway public key; one key serves all clients.
- **HMAC-SHA256:** cheaper on the client; each client holds a unique shared secret.
- Canonical encoding binds qname, qtype, answer, status, expiry and request ID, limiting substitution and replay.
- Key provisioning, rotation and revocation will be documented.

## Failure behaviour

- Reject a failed authenticator.
- Reject expired responses.
- Reject DNSSEC-*bogus* data.
- Never relabel *indeterminate* as valid; a strict policy fails closed and logs the reason.

# Optimization Proposed

## Move work off the constrained client while keeping verifiable status, freshness and a clear trust boundary.

| Dimension | Before (baseline) | After (LADG) | Measured by |
|---|---|---|---|
| DNSSEC computation | Client validates full chain, or uses non-validating DNS | Gateway validates; client verifies one compact authenticator | Client and gateway CPU time |
| Memory footprint | Validation logic and records burden the client | Client stores a small verifier + trust material | Peak RAM, firmware size |
| Repeated lookups | Repeated upstream queries | TTL-aware cache reuses valid results | Cache hit ratio, query count |
| Response authenticity | Answer may be accepted unvalidated | Authenticated envelope carries validation state | Tamper / spoof / replay tests |
| Freshness | Implementation-dependent | TTL, expiry and request binding checked | Expired-response rejection |
| Policy control | Resolver- or network-wide | Per-device policy and allowlist | Policy test coverage |
| Failure handling | Unclear on small clients | Explicit secure / bogus / insecure / indeterminate | Classification accuracy |

Benefits are hypotheses: cold- and warm-cache runs are measured separately, gateway overhead included, and no percentages are assumed before baselines.

# Solution Architecture & Design

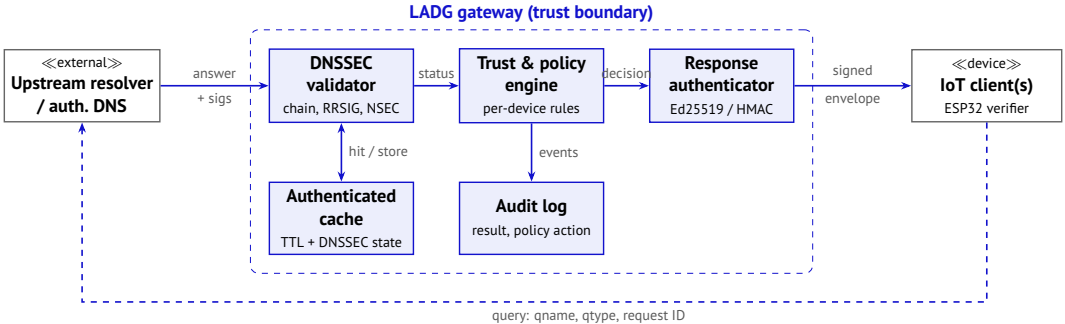



Figure: System Architecture UML Diagram

# Data Flow Diagram

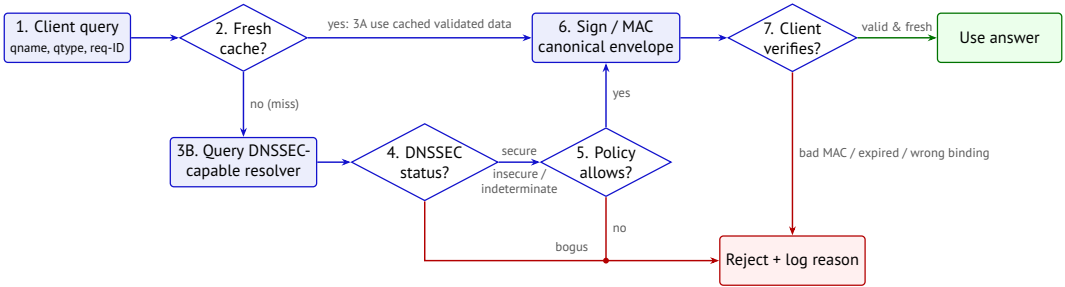



Figure: Data Flow Diagram (DFD)

# Data Flow Diagram: Key Data Objects

**Every object carries what the client needs to bind, verify and expire an answer, and nothing more.**

| Data object | Fields / purpose |
|---|---|
| Client query | Hostname, record type, client identity, request identifier |
| Validated cache entry | Answer, DNSSEC state, TTL, validation time, expiry, provenance |
| Gateway response envelope | Query binding, answer, status, expiry, request identifier, signature / MAC |
| Audit event | Timestamp, client, validation result, cache outcome, policy action (no unnecessary personal data) |

**Invalid (bogus) DNSSEC results are always rejected; indeterminate results follow explicit policy and are never marked valid.**

# Deployment Timeline

## Four short, testable stages, adjustable to the hackathon schedule.

| Milestone | Work items | Exit condition |
|---|---|---|
| **W1**: Research & design | Review DNSSEC RFCs; threat model, architecture, envelope fields, status semantics | Architecture and test plan approved |
| **W2**: Gateway prototype | Linux / Raspberry Pi gateway + validating resolver; query handling and status extraction | Lookups distinguish secure, bogus and indeterminate |
| **W3**: Client auth & cache | Signature / MAC, ESP32 client verifier, TTL-aware cache, device policies | Client rejects tampered, replayed and expired responses |
| **W4**: Test & evaluate | Valid, spoofed, modified, expired and resolver-failure scenarios; collect metrics | Reproducible results, demo, limitations and report |

# Deployment Timeline: Validation Methodology

**Each milestone is closed by functional, adversarial and performance tests in a controlled lab.**

- **Functional:** valid signed zone, bogus signature, insecure zone, non-existent name, expired cache entry.
- **Adversarial:** altered answer, forged gateway envelope, replay, wrong query binding, expired authenticator.
- **Performance:** conventional DNS vs. direct validation (where feasible) vs. gateway-assisted resolution.
- **Metrics:** latency, client and gateway CPU, peak RAM, upstream query count, cache hit ratio, response size.
- **Reproducibility:** software versions, configurations, run counts and test environment recorded.
- **Risk controls:** controlled test domains and an isolated lab network for attack simulation; private keys protected; resolver kept updated; limits of a compromised or unavailable gateway documented.

# Bibliography I

[1] R. Arends et al. “DNS Security Introduction and Requirements.” RFC 4033, IETF, March 2005.

[2] R. Arends et al. “Resource Records for the DNS Security Extensions.” RFC 4034, IETF, March 2005.

[3] R. Arends et al. “Protocol Modifications for the DNS Security Extensions.” RFC 4035, IETF, March 2005.

[4] P. Hoffman et al. “DNS Terminology.” RFC 8499, IETF, January 2019.

[5] Z. Hu et al. “Specification for DNS over Transport Layer Security (TLS).” RFC 7858, IETF, May 2016.

[6] D. Lawrence et al. “Serving Stale Data to Improve DNS Resiliency.” RFC 8767, IETF, March 2020.

[7] S. Josefsson and I. Liusvaara. “Edwards-Curve Digital Signature Algorithm (EdDSA).” RFC 8032, IETF, January 2017.

All RFCs available at https://www.rfc-editor.org/rfc/rfcNNNN

Thank You for Your Attention!